AI Ideas

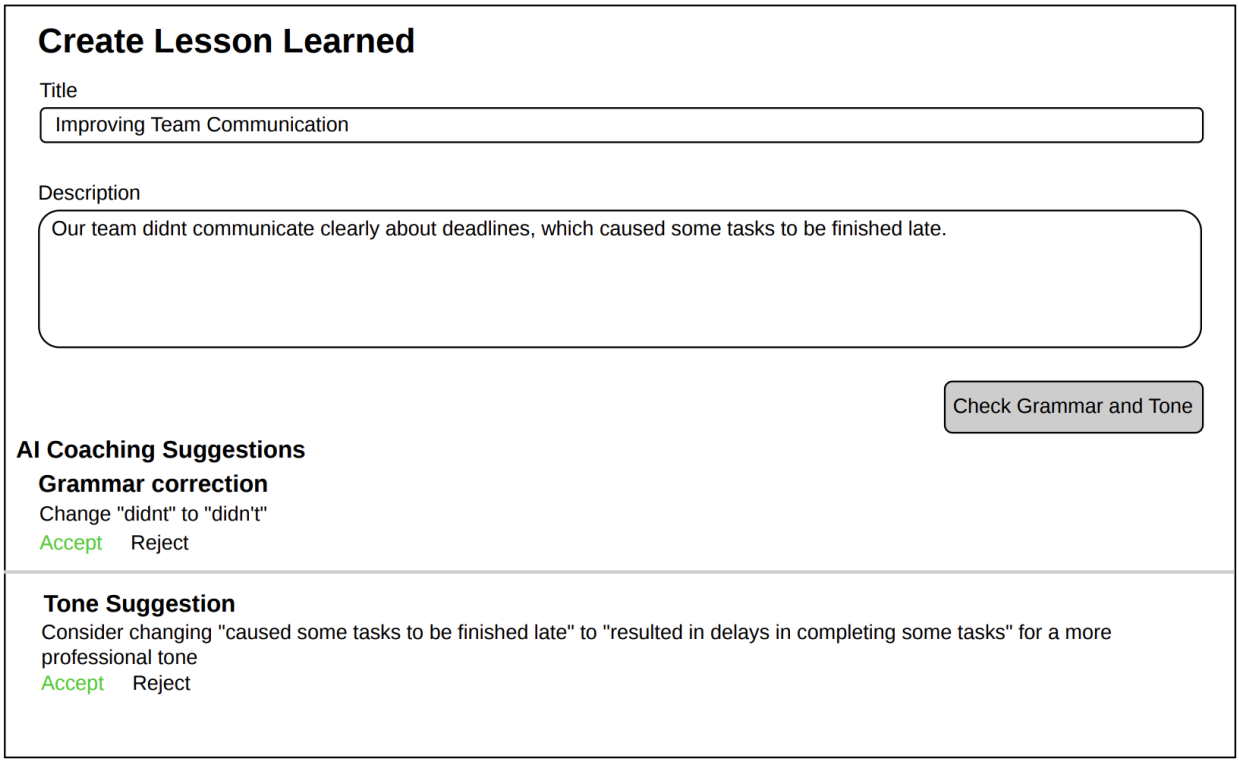
- Checking Grammar and Tone: Maleika
- Check for Required Information: Manuel/Kristena
- Title Generation: Sierra
- Keyword Generation: Sam

Input Fields and Prioritization:

- P1 - Lesson Title
- P2 - Lesson Description
- P3 - ‘Check Grammar and Tone” Button
- P3 - ‘Review Required Information’ Button
- P4 - ‘Generate Titles’ Button
- P5 - Keywords Field
- P5 - ‘Generate Keywords’ Button

Grammar and Tone

- **GUI Mockup**:



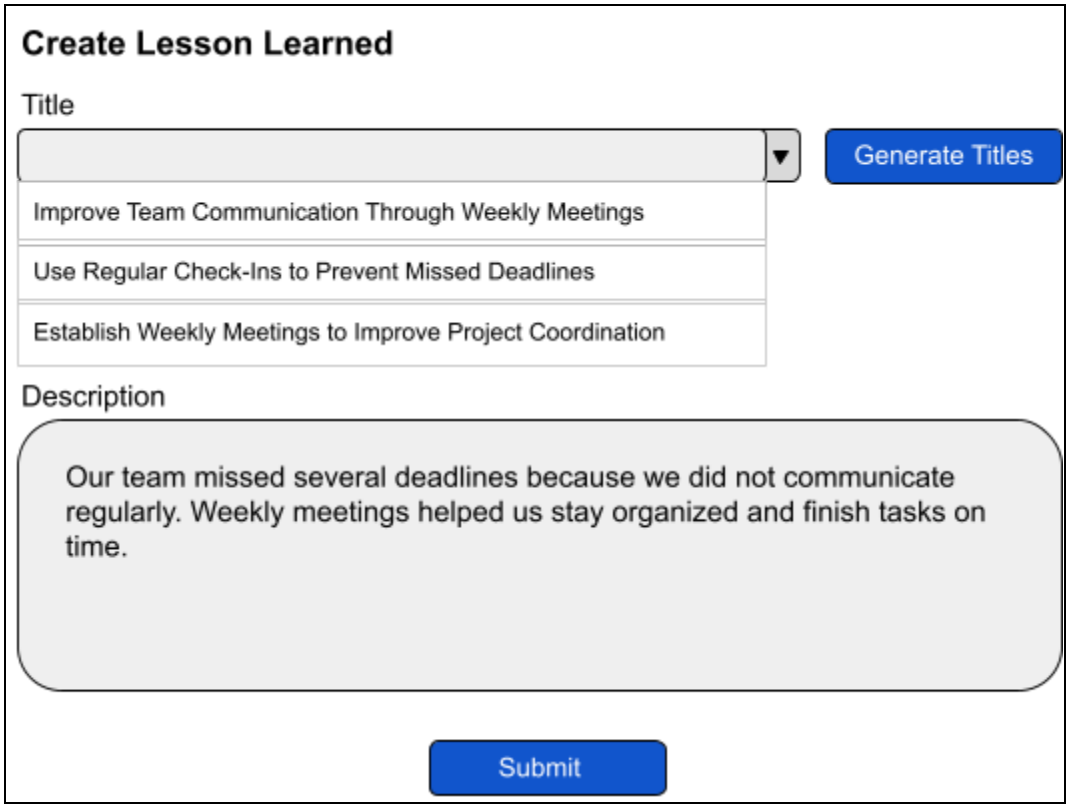
- **Concept of Operations**: When a contributor writes a lesson learned, an AI coach could check that lesson’s grammar and tone. This includes punctuation, spelling, and sentence structure, as well as flagging words or phrases that sound unprofessional or inappropriate. The AI coach then suggests improvements the user could make to their lesson by recommending alternative wording. The contributor can then decide whether they want to accept or reject each suggestion the AI coach makes.
- **List of Requirements**:

1. Check the spelling, punctuation, and grammar of a lesson learned.
2. Flag wording that may sound unprofessional or inappropriate for a lesson learned.
3. Check the writing style of a lesson learned for clarity and identify unclear or awkward sentences.
4. Suggest alternative wording for a lesson learned while preserving the author's original meaning.
5. Allow a contributor to accept or reject suggestions.

- **Potential Coaching Solutions**:
    1. A free-to-use AI tool that could be used to check the grammar of a lesson learned and suggest writing improvements is LanguageTool. LanguageTool is an AI-based grammar checker. The free version checks spelling, simple punctuation, and can correct some style mistakes. It also offers paraphrasing suggestions. One limitation is that LanguageTool's free version does not detect whether a lesson's tone is appropriate.
    2. A free-to-use tool that could be used to check the tone of a lesson learned is iTextwise. iTextwise is an AI-based tool that can rewrite text into multiple different tones, including professional, academic, technical and concise. The contributor could select between different tones and decide if they want to use the suggested rewrite for their lesson learned.

Title Generation

- **GUI Mockup**:



- **Concept of Operations**: The Automatic Title Generation feature analyzes the description entered by a contributor during lesson creation and suggests a short descriptive title that summarizes the main lesson learned. This will help contributors create a meaningful, professional title.
    a. Contributor enters the “Create Lesson” GUI
    b. Contributor enters a lesson description
    c. Contributor clicks “Generate Titles”
    d. Application sends the description to an AI model
    e. AI generates three potential titles
    f. Contributor writes their own title selects a title from the drop-down menu
    g. Contributor hits submit or edits the selected title and then hits submit
    h. Application validates the selected title and saves the lesson to the database

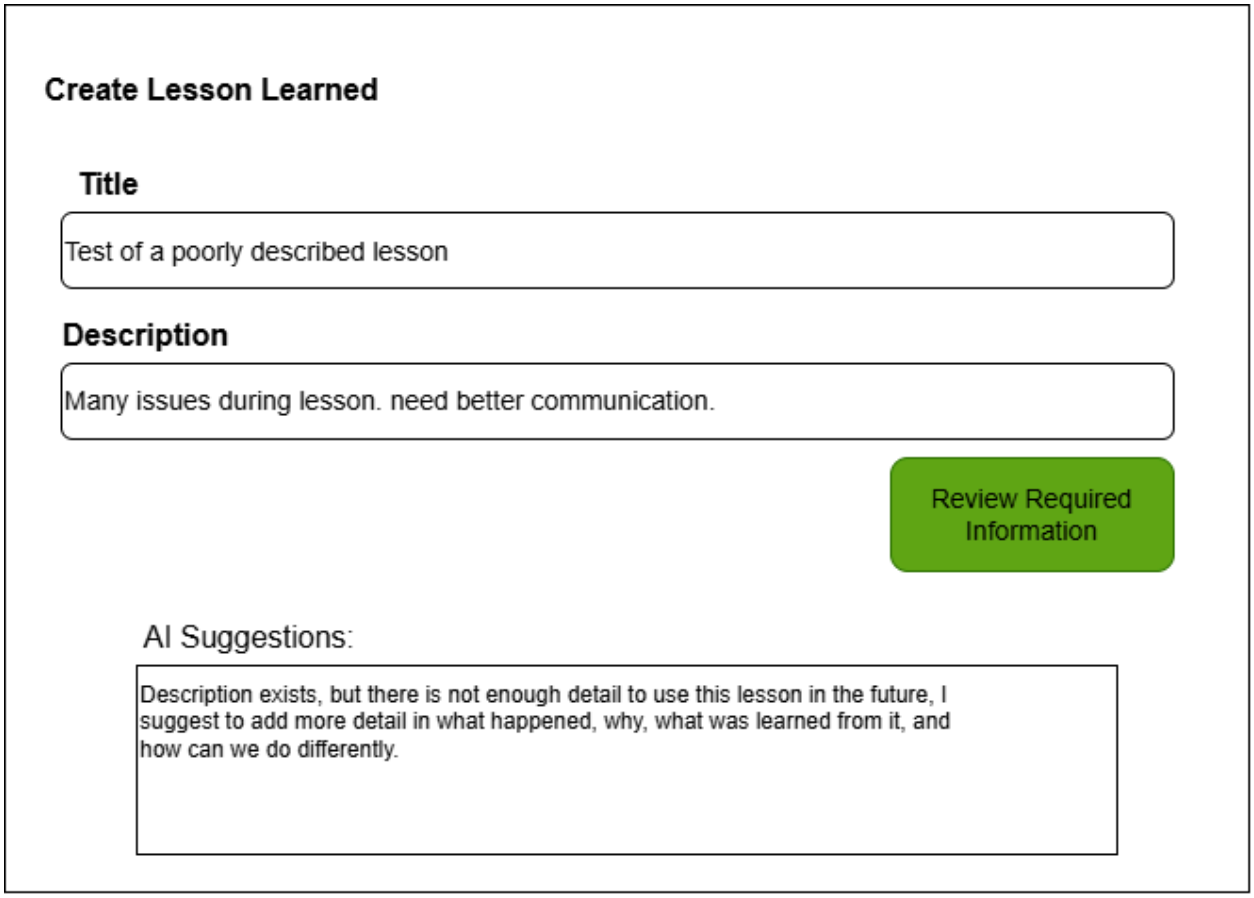
- **List of Requirements**:
    1. Generate a relevant title from a lesson description
    2. Keep titles concise, meaningful, and related to the description
    3. Allow contributors to accept, edit, regenerate, or ignore suggestions
    4. Validate generated title before saving a lesson to the database
    5. Generate multiple title suggestions
    6. Display helpful error when the description is empty or AI generation fails

- **Potential Coaching Solutions**:

- Ollama with Llama 3.2. Ollama can run a language model locally and provides an HTTP API that can be called by a Java application. It is free to run locally, but requires a license. Privacy is a non-issue since it is run locally.

Check for Required Information

- **GUI Mockup**:



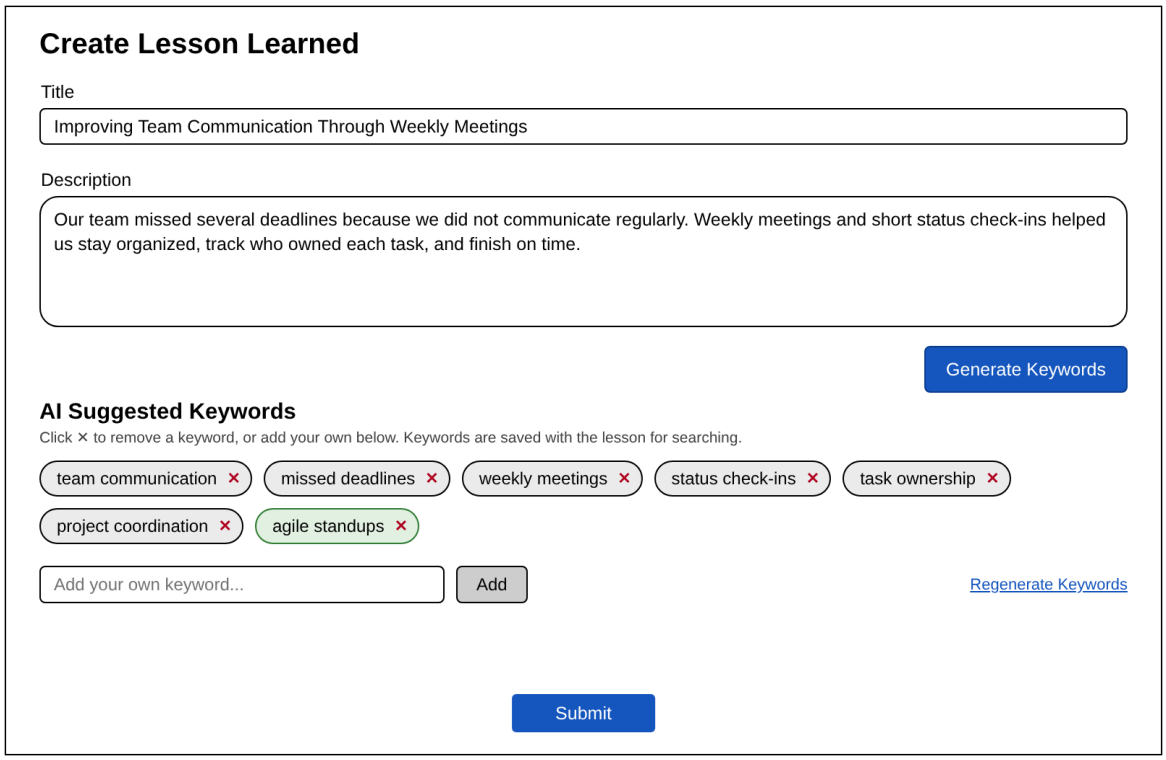
- **Concept of Operations**: because our system mostly checks for nonempty title and descriptions, We can make a Review Required Information button which will, on click, send the lessons title and description to the AI for review beside a lesson template we internally provide to the AI as a baseline, then the AI's response will appear in the "AI Suggestions" section near the bottom of the screen:
    1. Contributor enters description and/or title.
    2. Contributor clicks Review Required Information button
    3. Title and Description will be sent as JSON to AI alongside a baseline template or list of rules for a lesson to judge off of.
    4. AI returns in "AI Suggestions" with its opinion on how to improve the current lesson
    5. Contributor edits description and/or title based off suggestions
    6. Contributor clicks "Review Required Information" again to continue to refine the lesson until satisfaction is met.

- **List of Requirements**:
    1. Check if the description explains the situation, outcome, learned lesson, and recommendation.
    2. Identify missing or unclear information and explain what needs more clarification.
    3. Make sure AI is asking follow up questions and not “inventing” the missing facts.
    4. Allow Contributor to edit description and reattempt suggestions for more support.
    5. Show a message on empty input or if AI is unavailable.
- **Potential Coaching Solutions**:

Google AI Studio : Google AI Studio with Gemini could be used to prototype this coach. The contributor’s description would be provided with instructions to identify missing or unclear details and suggest follow-up questions. Google AI Studio supports testing prompts and system instructions, and the Gemini API offers a limited free tier. This AI also supports JSON input/output, so it will be much easier to pack the information into JSON format, then expect a JSON response to simply plug back into the GUI.

Keyword Generation

- **GUI Mockup**



- **Concept of Operations**

  Searching for lessons only works well when lessons are tagged with the words people will actually search for. A keyword generation coach reads the description a contributor wrote and suggests a short list of keywords and key phrases that capture what the lesson is about. The contributor reviews the list, removes any that don’t fit, adds their own, and saves the final keywords with the lesson. These keywords support the Phase 3

Viewer keyword search and the Curator epic of building search terms so users can find lessons even when their search words don't exactly match the lesson text.

A. Contributor enters the "Create Lesson" or "Update Lesson" GUI.
B. Contributor writes a lesson description.
C. Contributor clicks "Generate Keywords"
D. Application sends the description to Gemini and asks for 5-8 keywords returned as JSON.
E. Suggested keywords appear as chips in the keywords area.
F. Contributor removes unwanted keywords, adds their own, or regenerates.
G. Contributor clicks submit, and the application validates the keywords and saves them with the lesson.

- **List of Requirements**
  1. Generate 5-8 relevant keywords from a lesson description
  2. Only suggest keywords supported by the description, do not invent topics that are not in the lesson
  3. Remove duplicates and overly generic words
  4. Allow the contributor to accept, remove, add, or regenerate keywords before saving
  5. Validate keywords before saving
  6. Store Keywords as a new attribute of the LessonsLearned class so search() can match them.
  7. Display a helpful message when the description is empty or keyword generation fails
- **Potential Coaching Solutions**
  a. **Google AI Studio / Gemini API :** Gemini can read a lesson description and return keywords in a fixed JSON format. The application passes a JSON schema with the request and Gemini returns JSON that follows that schema, so the Java GUI can load the results. The schema can also cap how many keywords come back, since Gemini enforces the minimum and maximum item counts.
  Sources
  https://ai.google.dev/gemini-api/docs/structured-output
  https://ai.google.dev/gemini-api/docs/pricing